CHAPTER THREE

SYSTEM ANALYSIS AND DESIGN

3.1 Introduction

This chapter discussed advancements in artificial intelligence (AI) and machine learning, particularly the use of Convolutional Neural Networks (CNNs), as modern solutions to address the challenges in hair differentiation. By exploring various studies and methodologies, this chapter highlighted how AI-powered approaches have increased accuracy and reliability in classification, ultimately building consumer trust in hair products.

3.2 Conceptual Framework

Figure 3.1. above shows the conceptual outline for the development of the proposed model.

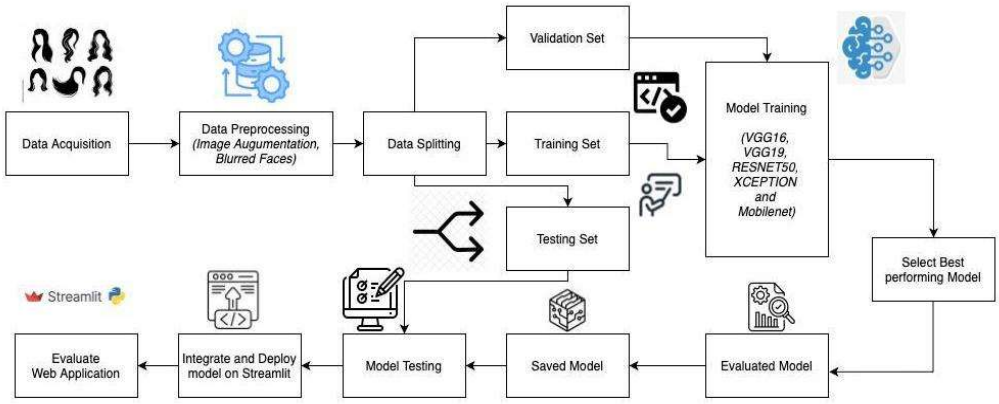

Figure 3.1: Conceptual framework of the system.

The process of distinguishing between human hair and synthetic hair began with data acquisition, where a large dataset of images of both human hair and synthetic hair was gathered. This dataset formed the foundation for training and validating the machine learning model. Preprocessing techniques such as rotation, scaling, and flipping were then applied to the data to enrich the dataset and boost the model's resilience by diversifying the training image collection. Additionally, in order to produce a better model, people's features were obscured. A training set, a validation set, and a testing set were created from the preprocessed data. In order to tune hyperparameters, the

model could be trained on one subset of the data, validated on another, and then tested on a different subset to assess its performance. The training set was used to train several deep learning models, including VGG16, VGG19, RESNET50, XCEPTION, and MobileNet. The models were chosen for proven architectures in image classification tasks. After training, the models were evaluated using the validation set to determine the accuracy and effectiveness in differentiating between human hair and synthetic hair. The model's performance was assessed, and the best-performing model was identified based on predefined metrics. The best-performing model was saved for deployment, having shown the highest accuracy and reliability during the validation phase. The saved model was further tested using the testing set to ensure its generalization capability and to validate its performance on unseen data. The validated model was then integrated into a web application using Streamlit, a popular framework for creating interactive web applications with Python. This step involved developing a user-friendly interface that allowed users to upload images and receive real-time classification results. The deployed web application was evaluated to ensure it met user requirements and provided accurate differentiation between human hair and synthetic hair. This step included user testing and feedback collection to refine the application. The process of distinguishing between human hair and synthetic hair involved several steps, including data acquisition, preprocessing, data splitting, model training, evaluated model, saved model, model testing, integrate and deploy model on Streamlit, and evaluate web application. The best-performing model was saved for deployment and further tested to ensure its generalization capability and accuracy.

3.3 Design Requirements

These requirements are specifications, criteria, or conditions that a product, system, or project must meet to fulfill its intended purpose. These requirements are essential for guiding the design process and ensuring that the final product meets the needs and expectations of users.

3.3.1 Functional Requirements

The following are the functional services rendered by the web application;

i. The application will support suitable image formats for processing.
ii. The application should be accessible on different browsers.
iii. The application should give good predictions and outputs.
iv. The application should display the results.

3.3.2 Non-Functional Requirements

These are requirements that characterize the operations of the application.

i. The system must deliver real-time and accurate predictions.
ii. The system must give timely results
iii. The system should be mobile-responsive
iv. The system should be user-friendly.

3.4 System Design

The system design for the hair classification project aimed to develop a deep learning model that accurately distinguished between human and synthetic hair using convolutional neural networks (CNNs). The process began with the acquisition of a diverse dataset of hair samples, including a wide range of hair types, lengths, and colors from various ethnicities and skin tones. The hair samples were then preprocessed, including resizing, normalizing, and potentially applying image augmentation techniques, to prepare the dataset for training the CNN model. The trained model was then evaluated using a test set to assess its performance in classifying hair samples, and the model was deployed.

3.4.1 Hair Quality Dataset

Gathering, storing, and annotating raw data stands as a cornerstone in supervised learning methodologies, constituting the initial and indispensable phase in crafting a robust dataset. For this project, the dataset was meticulously assembled through manual collection from internet searches and web sources, ensuring its suitability for model training purposes. Additionally, to maintain a consistent flow of training data and prevent the model from encountering image scarcity, a random function was implemented during training. This measure ensures continuous and successful training sessions.

3.4.2 Data Pre-processing

The goal of the pre-processing stage is to prepare the images for subsequent processing. Employing conventional filtering techniques enhances overall image resolution and boosts image quality. Initially, the images were resized to dimensions of 256×256 and 227×227 RGB format before being converted into NumPy arrays to ensure seamless compatibility with other Python libraries. Subsequently, normalization was applied by dividing each pixel value by 255, standardizing the numeric columns in the dataset. Following this, the images were associated with the corresponding classes, namely Human and Synthetic.

After these steps, a data generator was established. Given the image paths, it yielded batches of images along with respective labels [46].

3.4.3 CNN Layers Description

The CNN architecture comprises numerous layers, often referred to as multi-building blocks. Each model possesses its unique configuration of layers and distinct parameters that influence its performance results. The detailed composition of each layer within the utilized CNNs is delineated for every constructed network, with particular emphasis placed on the respective roles [47].

Figure 3.2 depicts a CNN architecture.

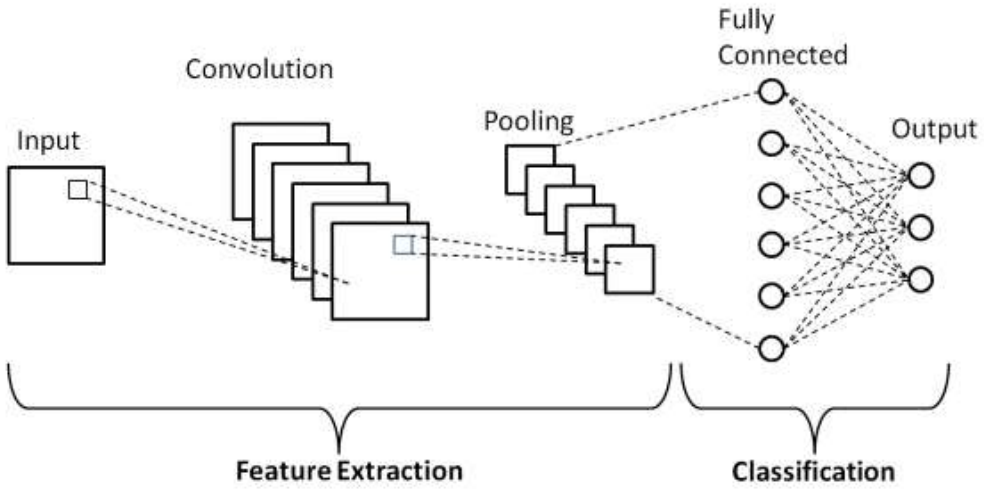

Figure 3.2: A diagram showing a CNN Architecture [9]

i. The **Convolutional Layer** comprises multiple convolutional filters, also known as kernels, which are applied to the input image to generate the output

feature map. Represented by Conv2D, this layer plays a pivotal role in feature extraction.

ii. In the **Pooling Layer,** large feature maps are condensed into smaller ones while retaining significant information throughout the pooling process. MaxPooling2D serves as the representation of this layer.

iii. Following the architecture of each network, the **Fully Connected layer** serves as the CNN classifier, situated after other layers. Each neuron in this layer connects to every neuron from the preceding layer. Typically represented by Dense, this layer facilitates classification, with each input node linked to each output case [47].

3.5 CNN Architectures Used

In the development of a hair quality detection system, several Convolutional Neural Network (CNN) architectures were employed to evaluate the performance and efficiency in distinguishing between human and synthetic hair. The architectures investigated included VGG16, VGG19, ResNet-50, Xception, MobileNet.

3.5.1 VGG (Visual Geometry Group) Networks

The VGG architectures, such as VGG16 and VGG19, are renowned for the depth achieved through the stacking of small convolutional filters (3x3).

This design allows for the extraction of detailed features from images, which makes it well-suited for tasks requiring high-resolution feature maps.

The formula for the output of each convolutional layer in these networks is:

$$F(x) = ReLU(Conv(x, W) + b)F(x) = ReLU(Conv(x, W) + b) \quad (3.1)$$

where the convolution operation Conv(x,W)Conv(x,W) with input xx and filter weights WW, along with the bias bb, captures the intricate textures and patterns that differentiate human hair from synthetic [24].

VGG16 is a deep learning model that excels in object detection and classification tasks. It is capable of accurately categorizing images into one of 1000 distinct categories with a remarkable 92.7% top-5 test accuracy on the ImageNet dataset. This model has gained significant popularity in the field of computer vision due to its simplicity and ease of use, particularly when leveraging transfer learning techniques.

Figure 3.3 and 3.4 shows the VGG-16 model architecture.

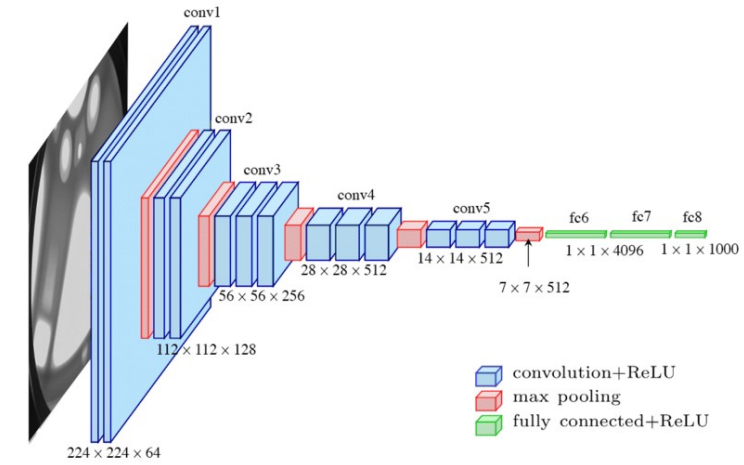

Figure 3.3: VGG-16 Model Architecture[49]

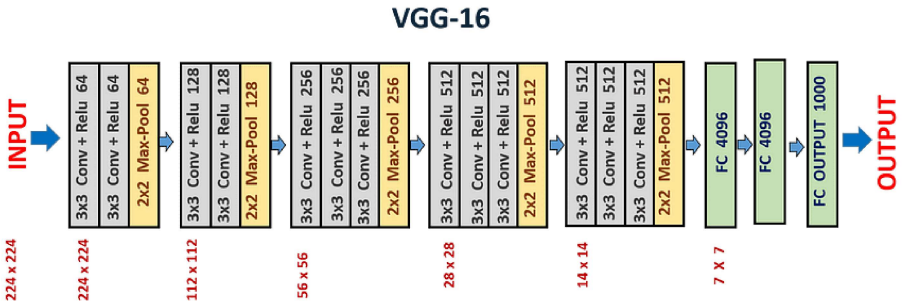

Figure 3.4: VGG-16 Model Architecture[50]

From Fig 3.4, the VGG16 model architecture consists of 16 layers with learnable parameters, despite having a total of 21 layers. This unique design allows for efficient processing and robust feature extraction. The model uses 3x3 convolutional filters with stride 1 and 2x2 max-pooling filters with stride 2 throughout, which enables it to learn rich features from images. The input to the model is a 224x224x3 tensor, and the convolutional layers have different numbers of filters: Conv-1 has 64 filters, Conv-2 has 128 filters, Conv-3 has 256 filters, and Conv-4 and Conv-5 have 512 filters. The fully connected layers are arranged hierarchically, with the first two layers having 4096 channels each and the third layer performing 1000-way classification with 1000 channels (one for each class). The final layer is the soft-max layer, which outputs the predicted class probabilities. This design enabled VGG16 to achieve a top-5 test

accuracy of 92.7% in the ImageNet Large Scale Visual Recognition Challenge (ILSVRC), making it a foundational architecture in the field of computer vision [49].

VGG-19 is a deep Convolutional Neural Network (CNN) model proposed by the Visual Geometry Group (VGG) from the University of Oxford. It is one of the models from the VGG family, which were introduced to investigate the effect of the depth of the network on its accuracy in large-scale image recognition tasks. VGG-19 stands out for its simple and homogeneous architecture, which consists of a sequence of convolutional layers followed by fully connected layers [51].

Figure 3.5 shows the model architecture of VGG-19.

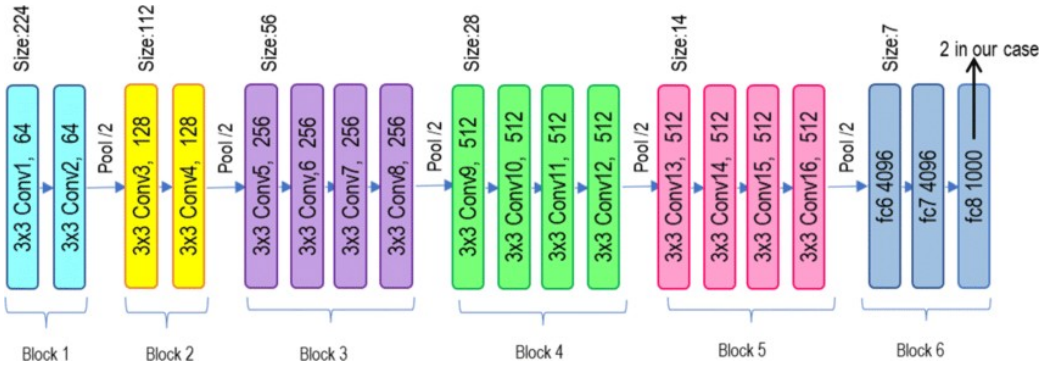

Figure 3.5: VGG-19 Model Architecture [52]

Fig 3.5 illustrates the VGG-19 model architecture which consists of 19 layers, including 16 convolutional layers, 3 fully connected layers, and 5 max-pooling layers. The convolutional layers use 3x3 kernels with a stride of 1 and employ spatial padding to preserve the spatial resolution of the input image. The max-pooling layers reduce the spatial dimensions of the feature maps by a factor of 2, effectively downsampling the input.A distinctive feature of the VGG-19 architecture is the use of the Rectified Linear Unit (ReLU) activation function, which introduces non-linearity and helps the model learn complex patterns more effectively compared to previous activation functions like sigmoid or tanh.VGG-19 has a large number of trainable parameters, approximately 143.67 million, which contributes to its impressive performance on image classification tasks. However, this also makes the model computationally intensive and resource-demanding, posing challenges for deployment on resource-constrained environments like mobile devices. The high computational requirements lead to significant storage needs, increased battery consumption, and potential latency issues, which are critical factors to consider in mobile applications [53].

3.5.2 ResNet50 ResNet (Residual Networks)

ResNet50 employs a residual learning framework, which introduces shortcut connections to facilitate the training of much deeper networks. This architecture is capable of extracting robust and highly discriminative features, essential for complex image classification tasks like hair quality assessment. The core formula in ResNet50's residual block is:

$$F(x) = ReLU(x + F(x, \{Wi\}))F(x) = ReLU(x + F(x, \{Wi\})) \quad (3.2)$$

where the residual function $F(x, \{Wi\})F(x, \{Wi\})$ typically involves a stack of two or three layers, enabling the network to learn more efficiently by bypassing the identity mapping [54].

Figure 3.6 shows the structure of the ResNet-50 architecture. It is majorly composed of input layer, multiple convolution layers, fully connected layers, and an output layer.

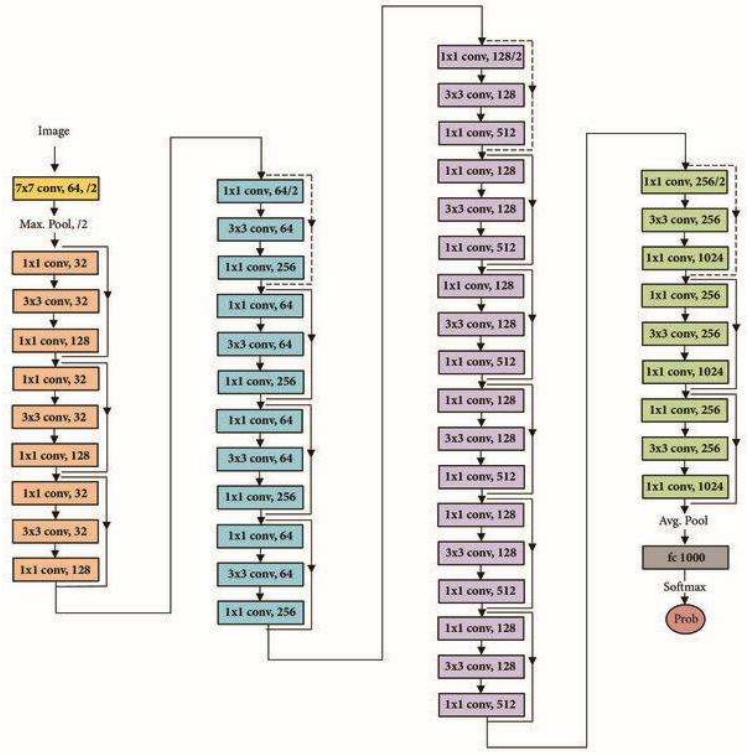

Figure 3.6: Structure of ResNet-50 [55]

ResNet was developed to address the degradation problem in deep neural networks, where performance degrades as the network becomes deeper. It solves this issue by

introducing residual blocks with skip connections, which allow gradients to flow directly across layers without being affected. This enables the training of deep networks, such as the 50-layer ResNet architecture, which employs bottleneck residual blocks stacked on top of each other [56].

Figure 3.7 shows the structure of a residual block in the ResNet architecture.

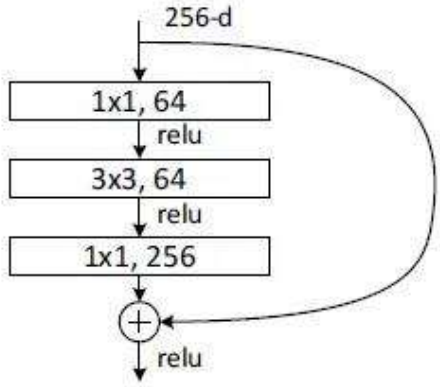

Figure 3.7: Structure of a Residual Block[56]

The components of the bottleneck residual block are:

i. Bottleneck Convolutional Layers:The block begins with a 1x1 convolutional layer to reduce the dimensionality of the input, followed by a 3x3 convolutional layer to extract features, and concludes with another 1x1 convolutional layer to restore the original number of channels.

ii. ReLU (Rectified Linear Unit): This non-linear activation function is applied after each convolutional layer, serving as the primary non-linearity in the architecture.

iii. Skip Connection: The most critical component of the residual block, the skip connection allows the input to be directly transferred to the output without any intermediate connections. This enables the preservation of important information from earlier layers without modification, facilitating the flow of gradients and improving training efficiency [55].

3.5.3 Xception (Extreme Inception)

Xception replaces traditional convolutional layers with depthwise separable convolutions, which significantly enhance computational efficiency while maintaining accuracy. This architecture is particularly useful for applications requiring detailed

feature extraction without compromising on speed, ideal for distinguishing subtle differences in hair texture.

The depthwise separable convolution in Xception can be expressed as:

$$F(x) = ReLU(DepthwiseConv(x, Wd) + bd + Conv(x, Wp) + bp) \quad (3.3)$$

where depthwise and pointwise convolutions together allow for efficient yet powerful feature learning [57].

The fundamental hypothesis behind Xception is that the mapping of cross-channel correlations and spatial correlations can be entirely decoupled. This architecture is composed of 36 convolutional layers structured into 14 modules, all of which have linear residual connections surrounding it, except for the first and last modules.

Xception uses a modified version of depthwise separable convolution, where the order is reversed compared to the standard approach. It performs pointwise convolution (1x1 convolution) first, followed by depthwise convolution. The architecture can be divided into three main parts: the entry flow, the middle flow (repeated 8 times), and the exit flow. All convolutional and separable convolutional layers are followed by batch normalization.

Compared to the Inception architecture, Xception takes a more "extreme" approach by entirely decoupling the task of cross-channel and spatial correlation learning, leading to improved performance on larger datasets like JFT. Xception has been shown to outperform models like VGG-16, ResNet, and Inception V3 in various image classification challenges, while being more efficient than the Inception architecture. Overall, the Xception architecture is a deep CNN model that relies on a novel depthwise separable convolution approach to decouple spatial and cross-channel correlations, resulting in an efficient and high-performing model for image classification tasks [57].

3.5.4 MobileNet

MobileNet leverages depthwise separable convolutions extensively, making it highly efficient and suitable for deployment on mobile and embedded devicesThe formula is akin to Xception's, emphasizing the balance between performance and computational load:

$$F(x) = ReLU(DepthwiseConv(x, Wd) + bd + Conv(x, Wp) + bp) \quad (3.4)$$

MobileNet models, which use a lightweight convolutional neural network architecture, significantly reduce the number of parameters and computational complexity compared to standard convolutional networks. This makes MobileNet more efficient and suitable for deployment on resource-constrained devices, such as mobile phones and embedded systems.

Figure 3.8 shows the structural module of MobileNet.

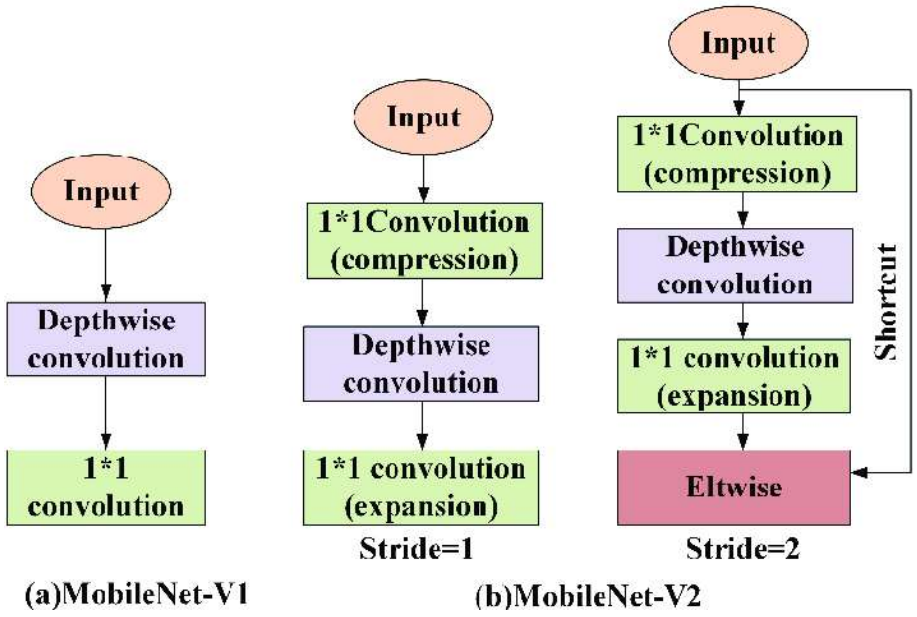

Figure 3.8: MobileNet Architectures [58]

MobileNet models come in different versions, with MobileNetV1, MobileNetV2, and MobileNetV3 being the main variants. Each version introduces improvements and optimizations over the previous one. For example, MobileNetV1 uses standard depthwise separable convolutions, while MobileNetV2 introduces the inverted residual structure with linear bottlenecks, and MobileNetV3 further optimizes the architecture for mobile devices. The MobileNet models can be easily scaled in terms of width and resolution, allowing for trade-offs between model size, latency, and accuracy depending on the target application.

MobileNet models are extensively used in many computer vision applications on embedded and mobile devices, including semantic segmentation, object identification, and picture classification. Because of its effective design, MobileNet models may be deployed on devices with limited resources, allowing for on-device inference and lowering the requirement for cloud-based processing—a critical feature for real-time

object identification and augmented reality applications. This efficiency does not come at the expense of accuracy, which is crucial for a real-time hair quality classification system that may operate on less powerful hardware [59].

3.5.5 Training Configurations

All models were trained using the RMSprop optimizer, binary cross-entropy as the loss function, and a set of predefined evaluation metrics (METRICS). This uniform training setup facilitates a fair comparison of model performance.

Each model employed ReLU (Rectified Linear Unit) activation functions within its hidden layers and a sigmoid activation function in the output layers.

The RMSprop optimizer was selected for its ability to adapt the learning rate dynamically, which is beneficial for handling the complexity and variability in the hair images. RMSprop helps in dealing with the varying magnitudes of parameter updates, providing stability during training.

Binary cross-entropy was used as the loss function to measure the performance of the models on the binary classification task.

Finally, a set of evaluation metrics (accuracy, precision, recall) was used to comprehensively assess the performance of each model. These metrics provided insights into how well the models distinguished between human and synthetic hair.

By training all models for 10 epochs under the same conditions, the project ensures fair and unbiased comparisons. This approach not only highlights the strengths of each architecture in the hair quality classification task but also provides insights into relative performance and computational efficiency.

3.5.6 Web Application

After evaluating the deep learning models (VGG16, VGG19, ResNet50, Xception, and MobileNet) for distinguishing between human and synthetic hair, the best-performing model was selected for deployment. Streamlit, an open-source app framework, was utilized to create an interactive web application to make this model accessible and easy to use. By deploying the best-performing model using Streamlit, the hair quality classification system became a practical and user-friendly application.

3.6 Algorithm

This algorithm augments a dataset of 276 images to improve model performance by generating diverse variations. Using "ImageDataGenerator," each image is transformed through rescaling, rotations, shifts, brightness adjustments, and random flips. These enhancements ensure a robust, varied dataset for more accurate model training.

Table 3.1: Algorithm for Data Augmentation

ALGORITHM 1: ALGORITHM FOR DATA AUGMENTATION	
1:	**Input:** Set of Images from the dataset, N: (number of images) = 276
2:	**Output:** Set of images in augmented dataset, N': (number of images)
3:	**Begin**
4:	**For** $\boldsymbol{i = 1\ to\ N}$
5:	Initialize the "ImageDataGenerator" function with the specified
6:	Rescale by 1/255 and rotate within the range $(-5^{\circ}, +5^{\circ})$.
7:	Shift horizontally by 20% of the total image width.
8:	Shift vertically by 20% of the total height.
9:	Apply shear transformation with shear angle of 0.2°.
10:	Adjust the brightness using a random factor within the range $(0.3, 1.0)$.
11:	Flip horizontally in a random manner
12:	Flip vertically in a random manner
13:	Set fill mode to "nearest"
14:	**END**

This algorithm applies various transformations to the input images to generate an augmented dataset, which can be used to improve the performance of the hair type classification model.

Table 3.2: Algorithm for the model development

ALGORITHM 2: ALGORITHM FOR THE MODEL DEVELOPMENT	
1:	**Input:** Training and validation datasets
2:	**Output:** Trained models and their performance metrics
3:	**BEGIN**
	//Model Configuration
4:	**For each model in {Xception, MobileNet, VGG16, VGG19, ResNet50}:**
5:	Load the pre-trained model with the following parameters:
6:	`input_shape=(224,224,3)`
7:	`include_top=False`
8:	`weights="imagenet"`
9:	**Initialize a Sequential model:**
10	Add the pre-trained model to the Sequential model.
11	Add a dropout layer with a rate of 0.5.
12	Add a flatten layer.
13	Add a batch normalization layer.
14	Add a dense layer with 32 units and 'he_uniform' kernel initializer.
15	Add another batch normalization layer.
16	Add a ReLU activation layer from equation 3.2.
17	Add another dropout layer with a rate of 0.5.
18	Add another dense layer with 32 units and 'he_uniform' kernel initializer.
19	Add another batch normalization layer.
20	Add another ReLU activation layer from equation 3.3.
22	Add another dropout layer with a rate of 0.5.
23	Add another dense layer with 32 units and 'he_uniform' kernel initializer.
24	Add another batch normalization layer.
25	Add another ReLU activation layer from equation 3.4.
26	Add a dense layer with 1 unit and sigmoid activation.
27	**NEXT**
28	**Compile the model with the following settings:**
29	Optimizer: 'rmsprop'
30	Loss function: Binary cross-entropy

31 | Metrics: Specified performance metrics
32 **NEXT**
32 **Initialize a learning rate reduction callback to monitor 'val_loss' with the following settings:**
33 | Patience: 3
34 | Verbose: 1
35 | Factor: 0.50
36 | Minimum learning rate: 1e-7
37 **NEXT**
38 Initialize a model checkpoint callback to save the best model based on 'val_accuracy'.
39 Initialize an early stopping callback with verbosity set to 1 and patience set to 3.
40 Train the model using the training and validation datasets, incorporating the learning rate reduction, model checkpoint, and early stopping callbacks.
41 **END.**

This algorithm is used to develop a model for hair quality detection. It involves loading pre-trained models such as Xception, MobileNet, VGG16, VGG19, and ResNet50, and then modification by adding layers such as dropout, flatten, batch normalization, and dense layers. The model is then compiled with the RMSprop optimizer, binary cross-entropy loss function, and specified performance metrics. The algorithm also includes callbacks for learning rate reduction, model checkpointing, and early stopping to monitor the model's performance during training.

3.7 Chapter Summary

The presented system effectively showcases an innovative approach for the identification and classification of hair quality through the utilization of artificial intelligence (AI). This method offers a distinctive solution for discerning various attributes and categorizing hair quality with precision and accuracy. Through the integration of advanced AI techniques, the system not only identifies but also categorizes hair quality, providing valuable insights for personalized styling recommendations, hair care analysis, and virtual try-on experiences.